Performance Metrics and Model Training Plan

Performance metrics:

1. Mean Absolute Error (MAE)

The first one is MAE which stands for Mean Absolute Error. So basically what this does is it takes every single prediction the model makes, looks at how far off it was from the actual real value, and averages all those differences out. The unit is micrograms per cubic metre which is just the standard unit for measuring air pollution concentration — so if our MAE comes out at 3.2 micrograms per cubic metre it means on any given day the model is typically about 3.2 micrograms per cubic metre wrong. To put that in context the WHO daily limit for PM2.5 is 15 micrograms per cubic metre so an error of 3.2 is roughly a 21% miss relative to that limit. This one is good because it's easy to explain to anyone — even people who aren't technical like policymakers or public health officials can understand what it means.

2. Root Mean Squared Error (RMSE)

The second is RMSE which is Root Mean Squared Error. This is similar to MAE but the key difference is it punishes bigger mistakes way more than smaller ones. So if the model is 3 micrograms per cubic metre off on a normal day that's not too bad, but if it's 30 micrograms per cubic metre off on a really high pollution day — like during a Saharan dust event or when traffic is really bad — that single massive error gets amplified a lot in the RMSE calculation. This matters a lot for our project specifically because being badly wrong on a dangerous pollution day has real consequences for people's health. One useful thing about RMSE and MAE together is that if they're close to each other it means the model is making fairly consistent errors, but if RMSE is much bigger than MAE it means the model is occasionally making some really large mistakes which we'd want to investigate.

3. R Squared (R^2)

The third is R squared. This one tells us what percentage of the variation in PM2.5 levels the model actually manages to explain. So PM2.5 goes up and down every day depending on traffic, weather, time of year and loads of other things — R squared tells us how much of that up and down the model successfully captures. A score of 1.0 would be perfect, 0 means the model is basically no better than just guessing the average every single day. Most papers doing similar PM2.5 forecasting work with LSTM and deep learning report R squared values somewhere between 0.75 and 0.95 so that gives us a realistic target to aim for. If Random Forest gets 0.78 and LSTM gets 0.89 that immediately shows LSTM is the better model for this task.

4. Mean Absolute Percentage Error (MAPE)

The fourth is MAPE which is Mean Absolute Percentage Error. Instead of giving the error in micrograms per cubic metre this one gives it as a percentage of the actual value. So if PM2.5 is 10 micrograms per cubic metre and the model predicts 13, the error for that day is 30%. This is useful because it puts the error in relative terms rather than absolute ones which makes it

easier to compare across different time periods. One thing to be aware of is MAPE can go a bit weird when the actual values are very close to zero because you're dividing by a tiny number, but for London Marylebone Road PM2.5 rarely drops that low so it shouldn't be a problem for us.

5. Mean Squared Error (MSE)

The fifth is MSE which is Mean Squared Error. This is basically RMSE before you take the square root. The reason we include this is that most neural networks including LSTM actually use MSE as their loss function during training — meaning it's what the model is trying to minimise when it's learning. So reporting MSE in our evaluation keeps things consistent with how the model was trained and it's also what most comparable papers report alongside RMSE.

6. Mean Bias Error (MBE)

The sixth is MBE which is Mean Bias Error. This one is different from all the others because it doesn't use absolute values — so positive and negative errors don't cancel each other out, they actually tell you something. A negative MBE means the model is consistently predicting lower than the actual values and a positive one means it's consistently predicting higher. For our project the negative case is the dangerous one — if the model keeps underpredicting pollution levels then health authorities might not trigger warnings when they should, people don't take precautions and that has real public health consequences. This is something neither MAE nor RMSE can tell you on their own which is why it's worth including.

Model Training Plan

Now for the model training plan with three of us each doing two models the split works out like this.

Person 1: Random Forest and XGBoost

The first person takes Random Forest and XGBoost. Random Forest is our baseline model — it builds loads of decision trees independently and averages their predictions. It's not designed for time series but it works really well when you give it good features like lag values, rolling averages and temporal encodings, and it also tells you which features are most important which is useful for the report. XGBoost is also tree based but instead of building trees independently it builds them one at a time where each new tree corrects the mistakes of the previous one. It usually performs better than Random Forest on structured data and is very fast to train. Comparing these two lets us see whether boosting or bagging gives better results within the tree-based family.

Person 2: LSTM and GRU

The second person takes LSTM and GRU. LSTM stands for Long Short-Term Memory and it's specifically designed for sequential data like time series. It has memory — it can remember

patterns from days or even weeks ago — which makes it much better at capturing things like a multi-day pollution build-up during a period of still weather. GRU is a lighter version of LSTM with fewer internal components so it trains faster and is less likely to overfit, especially on our dataset which isn't massive. Including both lets us see whether the extra complexity of LSTM is actually worth it or whether GRU gets you basically the same result with less effort.

Person 3: CNN-LSTM and Transformer

The third person takes CNN-LSTM and Transformer. CNN-LSTM combines a convolutional neural network for detecting short-term local patterns with LSTM for the longer term dependencies. So the CNN part might pick up on a 3-day spike that looks like a pollution episode and pass that information to the LSTM to put in context. The Transformer is a completely different kind of architecture that uses something called attention — instead of reading through the sequence step by step it looks at all past days at once and learns which ones are most relevant for predicting any given future day. This makes it particularly good for long horizon forecasting which is exactly what our project is about.